Transparency.dev Summit 2025

Gothenburg, Sweden | October 20–22

Implementing transparency in UEFI boot managers

Andrea Barisani, Daniele Bianco

```
REVERSEC FOUNDRY TEAM                                                    EXPERTISE
═══════════════════════════════════════════════════════════════════════════════════

┌─────────┐                                  ┌──────────────── research →  ┌─────────┬─────────┐
│ HARDWARE │──┐                              │                             │  RDS-TMC │ TEMPEST │
└─────────┘  │                              │                             │802.3 PIP │ EMV     │
             │                              │                             └─────────┴─────────┘
┌─────────┐  │   ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒   ├── penetration testing
│ FIRMWARE │──┼── SECURITY ──┤
└─────────┘  │   ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒   ├──────── code auditing
             │                              │
┌─────────┐  │                              └──────────── engineering →  ┌─────────┬─────────┐
│ SOFTWARE │──┘                                                          │USB ARMORY│ TAMAGO  │
└─────────┘                                                             └─────────┴─────────┘

HERITAGE
═══════════════════════════════════════════════════════════════════════════════════

2005 ─┬ inversepath.com                    ┌──────────────────┬──────────────────┐
2017  └─┬ foundry f-secure.com             │                  │  Daniele Bianco  │
2022    └─┬ foundry withsecure.com         │ Andrea Barisani  │ github.com/danbia │
2025      └── foundry.reversec.com         │                  ├──────────────────┤
                                           │   andrea.bio     │  Andrej Rosano   │
                                           │                  │ github.com/andrejro │
                                           └──────────────────┴──────────────────┘
                                           ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒
```

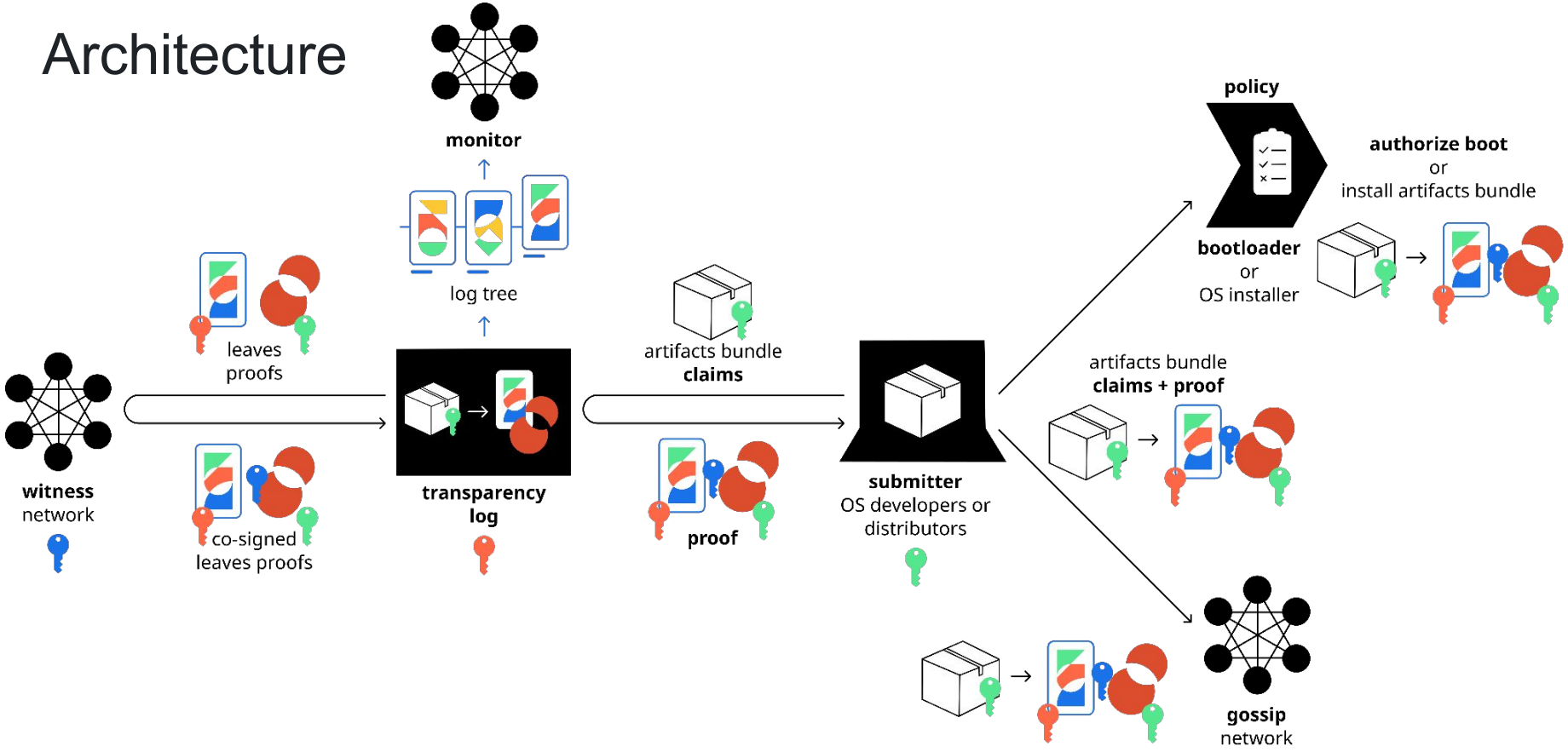
Introduction

The `boot-transparency` project is a Go library which enables interaction with transparency ecosystems for boot loading operations.

The goal is to enhance selection, authentication and policy enforcement on booted artifacts (e.g. kernels, UEFI binaries, ram disks).

The library is designed to be used also outside the bootloader context, for example imported from userspace kernel update tools, and supporting transparency proof verification on-line as well as off-line.

Architecture



Design

The `boot-transparency` library is designed with the following high-level goals:

- Make it easy to check a given boot policy
  - Support verification for the matching of the claimed data and the configured boot policy
  - Support the definition of signing quorums to authorize a given boot bundle
  - Support a built-in set of artifact categories that are commonly present in boot bundles
  - Enable support to expand the policy capabilities, by adding newer artifact categories in the future
- Enable support for multiple underlying transparency engines, (e.g. Sigsum and Tessera)
  - Support configuration for the transparency engine
  - Support configuration of transparency log, submitter and witness keys
- Make it easy to verify a given proof at transparency layer
  - Support (inclusion) proof verification
  - Support witness policy
  - The proof verification could be performed without having network access (e.g. bootloader).

Claimant model

Claim: OS vendor or developer claim the boot binaries are described by their manifest. The manifests contain a representative set of properties of the artifacts including a cryptographic hash of the binaries. The boot bundle includes one, or more artifacts. The artifacts are divided by category, as for example: `LinuxKernel`, `Initrd`, `Dtb`, `UEFIBinary`.

Statement: signed manifest including the claims for the boot bundle.

Claimant: the entity releasing, or distributing, the boot bundle (i.e. OS or UEFI binary vendor or developer).

Believer: the bootloader, or any user-space tool, interacting with the `boot-transparency` library to check transparency inclusion and validate matching between claims and requirements enforced by the boot policy. Based on such evaluations, the believers decide whether, or not, perform their Action.

Action: the bootloader will decide whether, or not, authorize the boot of the artifact bundle. The user-space tools will decide whether, or not, install the kernel/OS update.

Verifier: any entity that would check for any invalidation of the claims above, e.g.:

- OS vendors / developers ("has my identity been compromised?")
- other OS vendors
- AV/analysis companies/organisations with large security teams
- independent security researchers
- large organisations who already regularly look at software components for their fleet governments
- governments

Arbiters: there's no official body, but invalidated claims would affect reputation, possibly draw to further consequences not defined by this document.

Transparency engine

support for multiple
“transparency engines”

```go
type Engine interface {
    GetProof(proofBundle interface{}) ([]byte, error)
    ParseWitnessPolicy(wp []byte) (interface{}, error)
    SetKey(logKey []string, submitKey []string) error
    SetWitnessPolicy(wp interface{}) error
    ResetWitnessPolicy()
    VerifyProof(proofBundle interface{}) error
    ParseProof(jsonProofBundle []byte) (interface{}, []byte, error)
}
```

```go
type ProofBundle struct {
    // specify which transparency engine should be used to
    // verify this proof (i.e. Sigsum, Tessera)
    Format uint `json:"format"`
    // serialized JSON of Statement struct
    Statement []byte `json:"statement"`
    // serialized inclusion proof probing data,
    // its format depends by the chosen transparency engine
    Probe json.RawMessage `json:"probe,omitempty"`
    // inclusion proof, its format depends by the chosen transparency engine
    Proof json.RawMessage `json:"proof,omitempty"`
}
```

Sigsum transparency engine

```go
type Probe
type ProofBundle
type SigsumEngine
    func (e *SigsumEngine) GetProof(proofBundle interface{}) ([]byte, error)
    func (e *SigsumEngine) ParseProof(jsonProofBundle []byte) (interface{}, []byte, error)
    func (e *SigsumEngine) ParseWitnessPolicy(wp []byte) (interface{}, error)
    func (e *SigsumEngine) ResetWitnessPolicy()
    func (e *SigsumEngine) SetKey(logKey []string, submitKey []string) (err error)
    func (e *SigsumEngine) SetWitnessPolicy(wp interface{}) (err error)
    func (e *SigsumEngine) VerifyProof(proofBundle interface{}) (err error)
```

```go
type ProofBundle struct {
    Format    uint            `json:"format"`
    Statement json.RawMessage `json:"statement"`
    Probe     Probe           `json:"probe,omitempty"`
    Proof     string          `json:"proof,omitempty"`
}
```

```go
type Probe struct {
    // log origin
    Origin string `json:"origin"`
    // Sigsum uses leaf signature to identify the
    // leaf into the log
    LeafSignature string `json:"leaf_signature"`
    // log key hash in hex format as expected in Sigsum
    // proof bundle
    LogPublicKeyHash string `json:"log_public_key_hash"`
    // submitter key hash in hex format as expected
    // in Sigsum proof bundle
    SubmitPublicKeyHash string `json:"submit_public_key_hash"`
}
```

Proof of public logging

The transparency proof verification in the `boot-transparency` library is then summarized as follows:

1. **Compute checksum of the claim**.
2. **Check the keyhashes for both the log and leaf,** to ensure the correspondent log and submitter pubkeys are available.
3. **Check that the leaf signature** (with checksum computed as above) is valid.
4. **Check that the log's tree head signature** is valid.
5. **Verify all cosignatures** for witnesses known to the verifier.
6. **Verify the inclusion proof** is valid by computing the leaf hash from the checksum together with the keyhash and signature from the leaf line.

```
version=2
log=KEYHASH
leaf=KEYHASH SIGNATURE

size=NUMBER
root_hash=HASH
signature=SIGNATURE
cosignature=KEYHASH TIMESTAMP
SIGNATURE
cosignature=...

leaf_index=NUMBER
node_hash=HASH
node_hash=...
```

A very similar logic is illustrated in: [1] Sigsum - verifying a proof

[1] https://git.glasklar.is/sigsum/core/sigsum-go/-/blob/main/doc/sigsum-proof.md

Witness support

To enable witnessing, the `boot-transparency` library exposes a witnessing policy import functionality supporting the Sigsum policy format [1].

This format ensures compatibility with both Sigsum and Tessera.

```
log <key>
witness X1 <key>
witness X2 <key>
witness X3 <key>
group X-witnesses 2 X1 X2 X3

witness Y1 <key>
witness Y2 <key>
witness Y3 <key>
group Y-witnesses any Y1 Y2 Y3

group X-and-Y all X-witnesses Y-witnesses
quorum X-and-Y
```

[1] https://git.glasklar.is/sigsum/core/sigsum-go/-/blob/main/doc/policy.md

Artifact categories

`boot-transparency` natively supports a set of artifact categories that are commonly present in boot bundles.

At low-level, each category package registers an **artifact handler** that implements:

- the **parsing of requirements** for a given artifact. Such requirements can be specified in the boot policy to define authorization rules for a certain bundle of artifacts
- the **parsing of the claims** that are supported among the logged ones, for a given artifact
- the **functions to check** the matching between requirements and claims for a given artifact.

The library defines an high-level interface to the artifact handlers. This should allow to conveniently expand the policy capabilities, by adding newer artifact categories in the future.

```
const (
    // 0x0001 - 0x7FFF reserved for boot artifacts
    LinuxKernel uint = iota + 0x0001
    Initrd
    Dtb
    UEFIBinary
    WindowsBootMgr

    // 0x8001 - 0x8FFF reserved for bios artifacts
    UEFIBIOS

    // 0x9001 - 0x9FFF reserved for generic artifacts
    CVE
)


type Handler interface {
    // Parse serialized JSON containing requirements for
    // a given artifact
    ParseRequirements(jsonRequirements []byte) (interface{}, error)
    // Parse serialized JSON containing claims for
    // a given artifact
    ParseClaims(jsonClaims []byte) (interface{}, error)
    // Check matching between requirements and claims
    // for a given artifact
    Check(requirements interface{}, claims interface{}) error
}
```

Linux kernel claims and requirements

```go
type Claims struct {
    // filename of the artifact
    FileName string `json:"file_name,omitempty"`

    // SHA-512 hash of the artifact
    Hash string `json:"hash,omitempty"`

    // artifact version, using Semantic Versioning 2.0.0 (see semver.org)
    Version string `json:"version,omitempty"`

    // the architecture vocabulary is the one defined by the EFI specification
    // (i.e. IA32, x64, IA64, ARM, AA64, ...)
    Architecture string `json:"architecture,omitempty"`

    // true if the kernel is tainted
    Tainted bool `json:"tainted,omitempty"`

    // license(s) associated to this artifact.
    // Where applicable, licenses should be expressed as SPDX short-form IDs
    // (e.g.MIT, GPL-2.0-or-later, BSD-2-Clause)
    // https://spdx.github.io/spdx-spec/v2.3/SPDX-license-list/
    License []string `json:"license,omitempty"`

    // timestamp in RFC3339 format (e.g. "1985-04-12T23:20:50.52Z")
    // the claimant can decide to use this field to expose any relevant timestamp
    // for the artifact (e.g. the releasing date, tha building time, ...)
    Timestamp string `json:"timestamp,omitempty"`

    // public URLs to download the source code required to build the artifact
    SourceURLs []string `json:"source_urls,omitempty"`

    // serialized JSON containing arbitrary artifact information.
    // As an example, developers could include among metadata relevant arguments,
    // configuration flags, toolchain information, or any other detail used during
    // the building of the artifact
    Metadata string `json:"metadata,omitempty"`
}
```

```go
type Requirements struct {
    // required SHA-512 hash of the artifact
    Hash string `json:"hash,omitempty"`

    // required minimum version, expressed using Semantic Versioning 2.0.0
    MinVersion string `json:"min_version,omitempty"`

    // maximum allowed version, expressed using Semantic Versioning 2.0.0
    MaxVersion string `json:"max_version,omitempty"`

    // allowed architecture, the architecture vocabulary is the one defined by the EFI
    // specification (i.e. IA32, x64, IA64, ARM, AA64, ...)
    Architecture string `json:"architecture,omitempty"`

    // if true, tainted kernels are allowed
    Tainted bool `json:"tainted,omitempty"`

    // list of allowed licenses.
    License []string `json:"license,omitempty"`

    // allow only artifacts where the claimed timestamp is more recent than
    // the specified one in RFC3339 format (e.g. "1985-04-12T23:20:50.52Z")
    MinTimestamp string `json:"min_timestamp,omitempty"`

    // allow only artifacts that are claiming a given set of metadata (i.e. match check)
    Metadata string `json:"metadata,omitempty"`

    // allow only artifacts that are claiming a given set of metadata which is including
    // all the string(s) specified here (i.e. AND of inclusion checks)
    MetadataInclude []string `json:"metadata_include,omitempty"`

    // allow only artifacts that are claiming a given set of metadata which
    // is not including any of the strings specified here
    // (i.e. AND of negated inclusion checks)
    MetadataNotInclude []string `json:"metadata_not_include,omitempty"`
}
```

Boot policy - Linux

Authorize the boot only if the bundle includes:

- a **Linux kernel** that meets the following requirements:

  it is more recent than a certain version (e.g. `6.14.0-29`)

  it is **not tainted**

  it has been compiled with a given configuration option (`CONFIG_STACKPROTECTOR_STRONG=y`)

- an **init ram disk** that meets the following requirement:

  it is **not containing any tainted module**

- such artifact categories have been claimed in the log
- the claims have been signed by a sufficient number of trusted signers to **satisfy the required quorum** (e.g. **2**)

```
{
    "artifacts": [
        {
            "category": 1,
            "_comment": "0x0001 -> LinuxKernel",
            "requirements": {
                "min_version": "v6.14.0-29",
                "tainted": false,
                "metadata_include": [
                    "CONFIG_STACKPROTECTOR_STRONG=y"
                ]
            }
        },
        {
            "category": 2,
            "_comment": "0x0002 -> Initrd",
            "requirements": { "tainted": false }
        }
    ],
    "signatures": {
        "signers": [
            {
                "name": "signatory I",
                "pub_key": "ssh-ed25519 AAAAC3Nz...HDnK"
            },
            {
                "name": "signatory II",
                "pub_key": "ssh-ed25519 AAAAC3Nz...C49J"
            }
        ],
        "quorum": 2
    }
}
```

Boot policy - UEFI binary

Authorize the boot of a specific UEFI binary only if a given set of requirements on the booting system are met:

- the **UEFI binary hash** is matching the one claimed in the log
- the **UEFI BIOS** is more recent than a certain revision (e.g. **2.7.0**)
- the **UEFI BIOS** is provided by a certain vendor (e.g. **Lenovo**)

- the claims have been signed by a sufficient number of trusted signers to **satisfy the required quorum** (e.g. **2**)

```
{
    "artifacts": [
        {
            "category": 4,
            "_comment": "0x0004 -> UEFIBinary",
            "requirements": {
                "file_name": "systemd-bootx64.efi",
                "hash": "8ba6...eb59"
            }
        },
        {
            "category": 36864,
            "_comment": "0x9000 -> UEFIBIOS",
            "requirements": {
                "min_uefi_revision": "v2.7.0",
                    "firmware_vendor": [
                        "Lenovo"
                    ]
            }
        }
    ],
    "signatures": {
        "signers": [
            {
                "name": "signatory I",
                "pub_key": "ssh-ed25519 AAAAC3Nz...HDnK"
            },
            {
                "name": "signatory II",
                "pub_key": "ssh-ed25519 AAAAC3Nz...C49J"
            }
        ],
        "quorum": 2
    }
}
```

Boot policy - fallback initrd

Authorize the boot to every kernel, but only a given “trusted” init ram disk is allowed:

- the **fallback init ram disk hash** is matching the one claimed in the log
- the claims have been signed by a sufficient number of trusted signers to **satisfy the required quorum** (e.g. **2**)

```
{
    "artifacts": [
        {
            "category": 2,
            "_comment": "0x0002 -> Initrd",
            "requirements": {
                "file_name": "fallback-initrd.img-6.14.0-29-generic",
                "hash": "9f5d...b66c"
            }
        }
    ]
    "signatures": {
        "signers": [
            {
                "name": "signatory I",
                "pub_key": "ssh-ed25519 AAAAC3Nz...HDnK"
            },
            {
                "name": "signatory II",
                "pub_key": "ssh-ed25519 AAAAC3Nz...C49J"
            }
        ],
        "quorum": 2
    }
}
```

Tooling: bt-statement

```
$ ./cmd/bt-statement/bt-statement parse --statement=statement.json
{
  "description": "Linux bundle",
  "version": "v1",
  "artifacts": [
    {
      "category": 1,
      "claims": {
        "file_name": "vmlinuz-6.14.0-29-generic",
        "hash": "8ba6...eb59",
        "version": "v6.14.0-29-generic",
        "architecture": "x64",
        "tainted": false,
        "license": [
          "GPL-2"
        ],
        "metadata":"CONFIG_STACKPROTECTOR_STRONG=y"
      }
    },
    {
      "category": 2,
      "claims": {
        "file_name": "initrd.img-6.14.0-29-generic",
        "hash": "9f5d...b66c",
        "architecture": "x64",
        "tainted": false
      }
    }
  ]
}
```

```
$ ./cmd/bt-statement/bt-statement sign \
--statement=statement.json \
--signed-statement=signed-statement.json \
--private-key=keys/dist1-key
```

```
$ ./cmd/bt-statement/bt-statement verify \
--signed-statement=signed-statement.json \
--public-key=keys/dist1-key.pub
signature is valid
```

Allows to perform actions on the `boot-transparency` statement:

- parse
- sign
- verify

Tooling: bt-policy

```
$ ./cmd/bt-policy/bt-policy parse
--policy-file=./testdata/policy/policy.json
[
	{
		"artifacts": [
			{
				"category": 1,
				"requirements": {
					"min_version": "v6.14.0-29",
					"tainted": false,
					"architecture": "x64"
				}
			},
			{
				"category": 2,
				"requirements": {
					"architecture": "x64",
					"tainted": false
				}
			}
		],
		"signatures": {
			"signers": [
				{
					"name": "signatory I",
					"pub_key": "ssh-ed25519 AAAA…"
				}, ...
			],
			"quorum": 2
		}
	}
  ]
```

```
$ ./cmd/bt-policy/bt-policy check \
--policy-file=policy.json \
--signed-statement=signed-statement.json
signed statement is matching the policy
```

Allows to perform actions on the `boot-transparency` policy:

- parse
- check against a given statement

Tooling: bt-bundle

```
$ ./bt-proof-bundle create
Usage: ./bt-proof-bundle create [-h] [-e transparency-engine] [-i
inclusion-proof-file] [-p probe-file] [-s signed-statement-file]
[parameters ...]
 -e, --engine=transparency-engine
             Transparency engine (i.e. 1: Sigsum or 2: Tessera)
             (required)
 -h, --help  Show usage message and exit
 -i, --inclusion-proof=inclusion-proof-file
             Inclusion proof file (required)
 -p, --probe=probe-file
             Proof probe file, contains data to request the inclusion
             proof to the log (required)
 -s, --signed-statement=signed-statement-file
             Signed statement file (required)
```

```
$ ./cmd/bt-proof-bundle/bt-proof-bundle create \
--engine=1 \
--inclusion-proof=signed-statement.json.proof \
--probe=probe.json \
--signed-statement=signed-statement.json  > bt-proof-bundle.json
```

```
$ ./bt-proof-bundle parse --bundle=bt-proof-bundle.json
-e 1
{
  "format": 1,
  "statement": {
    "description": "Linux bundle",
    "version": "v1",
    "artifacts": [
      {
        "category": 1,
        "claims": {
        "file_name": "vmlinuz-6.14.0-29-generic",
        "Hash": "8ba6...eb59",
        "version": "v6.14.0-29-generic",
        "tainted": false,
      }, …
    ]
  }
  "probe": {
    "origin": "https://test.sigsum.org/barreleye",
    "leaf_signature": "0918 ... ff00",
    "log_public_key_hash": "4e89 ... bb4d",
    "submit_public_key_hash": "3029 ... 765c"
  },
  "proof": "version=2\nlog=4e89 ... 9c36\n"
}
```

Tamago in a nutshell

- A **minimally**[1] patched Go distribution to enable GOOS=tamago support, which provides freestanding execution on GOARCH={**amd64**|arm|**arm64**|riscv64} bare metal.
- A set of packages[2] for platform support, including hardware initialization (e.g. boot).

AMD64	
cloud_hypervisor/vm	Cloud Hypervisor
firecracker/microvm	Firecracker
qemu/microvm	QEMU microvm
uefi/x64	UEFI
ARM	
nxp/mx6ullevk	NXP MCIMX6ULL-EVK
raspberrypi/pi1	Pi 1
raspberrypi/pi2	Pi 2
raspberrypi/pizero	Pi Zero
usbarmory/mk2	USB armory Mk II (USB+LAN)
RISCV64	
qemu/sifive_u	QEMU SiFive HiFive Unleashed

1 https://github.com/usbarmory/tamago-go
2 https://github.com/usbarmory/tamago

Tamago & go-boot

A TamaGo unikernel implementing a **UEFI Shell** and **OS loader** for AMD64 platforms.

It supports loading of arbitrary EFI images, Linux kernels and Windows UEFI boot manager.

It provides a clean pure Go API to UEFI Boot and Runtime services in ~2000 LOC.

It results in a 3MB EFI application which performs as fast, if not faster, than equivalent functionality Implemented in other C/Rust based efforts.

It supports **UEFI networking** through SNP, bringing full Go network I/O in your pre-boot environment.

https://github.com/usbarmory/go-boot
https://github.com/usbarmory/go-boot/wiki

```
initializing EFI services
initializing console (text)

go-boot • tamago/amd64 (go1.24.2) • UEFI x64

.                <path>                    # load and start EFI image
build                                      # build information
cat              <path>                    # show file contents
clear                                      # clear screen
cpuid            <leaf> <subleaf>          # show CPU capabilities
date             (time in RFC339 format)?  # show/change runtime date and time
exit,quit                                  # exit application
halt,shutdown                              # shutdown system
info                                       # device information
linux,l,\r       (loader entry path)?      # boot Linux kernel image
log                                        # show runtime logs
lspci                                      # list PCI devices
memmap           (e820)?                   # show UEFI memory map
mode             <mode>                    # set screen mode
peek             <hex offset> <size>       # memory display (use with caution)
poke             <hex offset> <hex value>  # memory write   (use with caution)
protocol         <registry format GUID>    # locate UEFI protocol
reset            (cold|warm)?              # reset system
stack                                      # goroutine stack trace (current)
stackall                                   # goroutine stack trace (all)
stat             <path>                    # show file information
uefi                                       # UEFI information
uptime                                     # show system running time
windows,win,w                              # launch Windows UEFI boot manager

> uefi
UEFI Revision ......: 2.70
Firmware Vendor ....: EDK II
Firmware Revision ..: 0x10000
Runtime Services  ..: 0xbf9ecb98
Boot Services ......: 0xbfea5720
Frame Buffer .......: 1280x800 @ 0xc0000000
Configuration Tables: 0xbf9ecc98
  ee4e5898-3914-4259-9d6e-dc7bd79403cf (0xbf8ebe98)
  05ad34ba-6f02-4214-952e-4da0398e2bb9 (0xbfea58a0)
  7739f24c-93d7-11d4-9a3a-0090273fc14d (0xbf8e7018)
  4c19049f-4137-4dd3-9c10-8b97a83ffdfa (0xbfea54e0)
  49152e77-1ada-4764-b7a2-7afefed95e8b (0xbfea6180)
  060cc026-4c0d-4dda-8f41-595fef00a502 (0xbf940018)
  eb9d2d31-2d88-11d3-9a16-0090273fc14d (0xbf93f000)
  eb9d2d30-2d88-11d3-9a16-0090273fc14d (0xbfb7e000)
  8868e871-e4f1-11d3-bc22-0080c73c8881 (0xbfb7e014)
  dcfa911d-26eb-469f-a220-38b7dc461220 (0xbebb6018)
```

Less is more

Bringing bare metal Go to pre-boot UEFI applications allows overall reduction of UEFI usage, while benefiting from its boot services and runtime context.

~~UEFI TCPv4/TCPv6~~	replaced by gvisor over Simple Network Protocol
~~UEFI TLS, HTTP/HTTPS~~	replaced by Go net/tls, net/http state-of-the-art implementation
~~UEFI Regular Expression~~	replaced by Go regexp
~~UEFI Byte Code~~	replaced by wazero
~~UEFI IPSec~~	replaced by Tailscale

We spare UEFI from complex, potentially unsafe, least performant and potentially out-of-date protocols, favoring **modern**, **secure**, **application/user controlled** stacks.

The burden placed towards UEFI are reduced, without compromises, restoring a more favorable separation of roles (and to a certain audience confidence in UEFI overall).

Boot-transparency integration (aka cut&paste)

Files

boot-transpare...

Go to file t

- cmd
  - amd64.go
  - common.go
  - debug.go
  - linux.go

go-boot / cmd / linux.go

Code Blame 460 lines (369 loc) · 11.8 KB Raw

```go
func bootTransparencyOfflineCheck(fsys fs.FS, bootPolicyPath string, witnessPolicyPath string, submitKeyPath string, logKeyPath string, proofBundlePath string) (err error) {
    bootPolicy, err := fs.ReadFile(fsys, bootPolicyPath)
    if err != nil {
        return fmt.Errorf("cannot read boot policy, %v", err)
    }

    witnessPolicy, err := fs.ReadFile(fsys, witnessPolicyPath)
    if err != nil {
        return fmt.Errorf("cannot read witness policy, %v", err)
    }

    // select Sigsum as transparency engine
    te, err := transparency.GetEngine(transparency.Sigsum)
    if err != nil {
        return fmt.Errorf("unable to configure the transparency engine: %v", err)
    }

    // set public keys
    err = te.SetKey([]string{string(logKey)}, []string{string(submitKey)})
    if err != nil {
        return err
    }

    // parse witness policy
    wp, err := te.ParseWitnessPolicy(witnessPolicy)
    if err != nil {
        return err
    }

    // set witness policy
    err = te.SetWitnessPolicy(wp)
    if err != nil {
        return err
    }

    // parse the proof bundle, which is expected to contain
    // the logged statement and its inclusion proof
    pb, _, err := te.ParseProof(proofBundle)

    // inclusion proof verification
    // considers the co-signing quorum as defined in the witness policy
    err = te.VerifyProof(pb)
    if err != nil {
        return err
    }
```

A “crude” example of (one of) the advantages of using Go to extend bootloader functionalities.

The **same code cut&pasted from the** `boot-transparency` **“stand-alone” example**:

https://github.com/usbarmory/boot-transparency/blob/main/example/check-bundle/check-bundle.go

...and **smashed into go-boot linux command**.

Future improvements

Monitoring / **Gossiping**:

- Support the concept of **advanced monitors** that can generate **alarms in form of artifacts**, or more in general, publish artifacts that could enhance to the “original” proof bundle by adding further information on the top of the claims made by the distributor.

  Such information would be consumed by bootloaders, or installers, with network access, and accounted during the policy evaluation process (e.g. refuse to authorize bundles where one or more inconsistencies have been highlighted in the original claims, refuse to authorize bundles based on an updated list of CVEs affecting the artifacts included in the bundle).

- Evaluate strategies to integrate the **reproducible builds** concept.

  This would require extending the current “metadata” claim into something “more structured” to allow advanced monitors, possibly based on existing tools, to spot out inconsistencies across a group of independent builders analyzing the same bundle of artifacts.

Transparency log:

- Improve support for **multiple log sources.**

General:

- Overall **codebase cleanup** and **bugfixing**.

Boot-transparency

Repository: https://github.com/usbarmory/boot-transparency
Documentation: https://github.com/usbarmory/boot-transparency/wiki

go-boot

Repository: https://github.com/usbarmory/go-boot
HCL: https://github.com/usbarmory/go-boot/wiki

TamaGo

Repository: https://github.com/usbarmory/tamago
Documentation: https://github.com/usbarmory/tamago/wiki
API: https://pkg.go.dev/github.com/usbarmory/tamago
Example: https://github.com/usbarmory/tamago-example

USB armory

Repository: https://github.com/usbarmory/usbarmory
Documentation: https://github.com/usbarmory/usbarmory/wiki
HAB/OTP tool: https://github.com/usbarmory/crucible

GoTEE

Repository: https://github.com/usbarmory/GoTEE
Documentation: https://github.com/usbarmory/GoTEE/wiki
Example: https://github.com/usbarmory/GoTEE-example

Armored Witness

Repository: https://github.com/transparency-dev/armored-witness
Bootloader: https://github.com/transparency-dev/armored-witness-boot
OS: https://github.com/transparency-dev/armored-witness-os
Applet: https://github.com/transparency-dev/armored-witness-applet

Andrea Barisani

andrea.barisani@reversec.com

Daniele Bianco

daniele.bianco@reversec.com

https://foundry.reversec.com